Performance Evaluation of Trie and Suffix Array Based Text Searching Techniques

Rajni Tiwari
CMR Institute of Technology
Bengaluru, Karnataka, India
rajni.t@cmrit.ac.in

Debasmita Mishra
Presidency University
Bengaluru, Karnataka, India
debasmita.mishra@presidencyuniversity.in

Dr.Srabana Pramanik
Presidency University
Bengaluru, Karnataka, India
srabana.pramanik@presidencyuniversity.in

Neha Arora
Presidency University
Bengaluru, Karnataka, India
neha.arora@presidencyuniversity.in

Priyanka Niranjan Savadekar
Presidency University
Bengaluru, Karnataka, India
priyanka@presidencyuniversity.in

***Abstract*—Efficient handling of repeated pattern matching queries is a critical requirement in systems that manage large and relatively static text collections. Conventional sequential scanning techniques, while easy to implement, exhibit rapidly increasing response times as dataset size and query frequency grow.**

This study presents an experimental evaluation of three commonly used text searching approaches: naive sequential search, Trie-based prefix indexing, and Suffix Array-based substring indexing. All methods are implemented under identical conditions to ensure a fair comparison, with performance assessed in terms of query execution time, preprocessing overhead, and memory consumption across varying input sizes.

Experimental results demonstrate that index-based techniques provide substantial improvements in search latency compared to scan-based methods when multiple queries are executed on the same dataset. Trie-based indexing achieves the lowest response times for prefix-oriented queries, whereas Suffix Arrays offer greater flexibility for general substring searches at the cost of moderate preprocessing and memory overhead.

The findings highlight practical trade-offs among the evaluated techniques and provide implementation-oriented insights for selecting appropriate text search strategies in real-world applications.

Keywords— ***Text Searching, Trie, Suffix Array, String Matching, Performance Evaluation***

I. Introduction

The rapid growth of digital information has led modern computing systems to manage and process increasingly large volumes of textual data. Sources such as system logs, online documents, software repositories, and analytical records generate text collections that are often queried repeatedly while undergoing minimal structural change. In such environments, the efficiency of search operations plays a crucial role in determining overall system responsiveness and user experience.

In many practical scenarios, the cost of executing repeated search queries outweighs the one-time cost of preparing the data. As a result, search performance is governed not only by algorithmic complexity but also by how effectively the underlying data representation supports repeated access. Designing search mechanisms that deliver consistent and predictable response times therefore remains an important challenge in large-scale text processing systems.

A commonly adopted baseline approach for pattern matching is sequential or naive scanning, in which the input text is examined at every possible alignment position to locate a target pattern. This technique is appealing due to its conceptual simplicity and negligible preprocessing requirements. However, its performance degrades rapidly as dataset size and query frequency increase, since each query requires a full traversal of the text. Consequently, sequential scanning becomes unsuitable for applications that demand scalability and stable response times.

To overcome these limitations, indexing-based text search strategies have been widely explored. Prefix-oriented indexing structures, most notably Tries, organize textual data by exploiting shared prefixes among strings. This organization enables search operations whose execution time depends primarily on the length of the query rather than the size of the dataset. Such characteristics make Trie-based techniques particularly effective for prefix-dominated workloads, including dictionary lookup and autocomplete systems.

Suffix-based indexing approaches extend search capabilities by enabling efficient substring matching. Among these, Suffix Arrays represent a space-efficient alternative to suffix trees by storing sorted references to suffix starting positions within the text. By transforming substring search into a bounded comparison problem, Suffix Arrays support flexible pattern queries while maintaining moderate memory overhead. The construction of such indices introduces additional preprocessing cost, but this cost can be amortized across multiple queries in static or semi-static datasets.

Although these techniques are well established, existing research often emphasizes theoretical complexity analysis or evaluates individual algorithms in isolation. Comparative studies that implement multiple search strategies using the same datasets, programming environment, and evaluation metrics are relatively limited. This lack of uniform experimental evaluation makes it difficult to assess practical trade-offs related

to runtime behavior, scalability, and memory usage under real-world constraints.

This work addresses this gap by presenting a controlled experimental comparison of sequential scanning, Trie-based prefix indexing, and Suffix Array-based substring indexing. By maintaining consistent implementation choices and experimental conditions, the study aims to provide an objective assessment of how these techniques perform as input size and query workload increase.

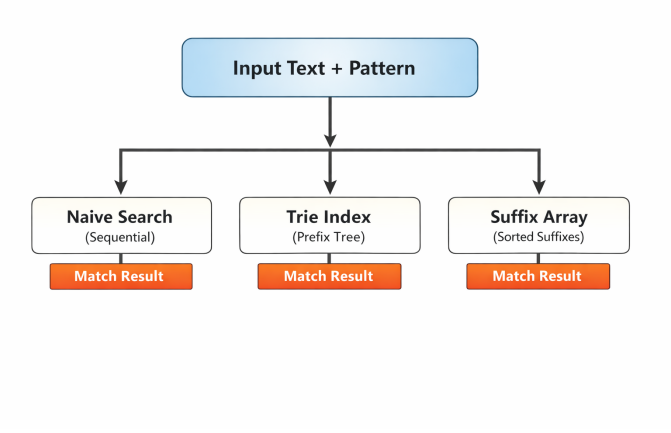

Fig. 1. Conceptual overview of sequential scanning, Trie-based prefix search, and Suffix Array-based substring search

A. Novelty and Contribution

The key contribution of this study lies in its uniform and implementation-focused comparison of three representative text searching techniques that reflect different levels of algorithmic sophistication. Unlike many prior works that focus exclusively on theoretical bounds or optimized variants of a single approach, this study evaluates naive sequential search, Trie-based indexing, and Suffix Array-based indexing within a shared experimental framework.

All techniques are implemented using the same programming language, executed on identical datasets, and evaluated using consistent performance metrics. This controlled setup enables direct observation of trade-offs among preprocessing overhead, query execution time, memory consumption, and scalability behavior, isolating the impact of the underlying data structures rather than external implementation factors.

II. Related Work

Efficient string matching and text search have been long-standing research topics due to their central role in information retrieval, data analysis, and software systems. In environments where large text collections are queried repeatedly, the effectiveness of a search strategy has a direct impact on system throughput and latency. As a result, a wide range of approaches have been proposed to balance execution speed, memory usage, and preprocessing overhead.

A. Scan-Based and Baseline Matching Techniques

Early solutions to the text searching problem relied on direct comparison between the query pattern and the input text at successive alignment positions. These scan-based methods require minimal preprocessing and additional storage, making them suitable for small datasets or one-time queries. However, their performance degrades linearly with input size, and repeated queries incur redundant computation. Numerous studies have demonstrated that such approaches fail to scale in systems that demand predictable response times under growing workloads.

Several algorithmic improvements, including heuristic skipping and early mismatch detection, have been proposed to mitigate the cost of sequential scanning. While these enhancements reduce average-case execution time in specific scenarios, they do not eliminate the fundamental dependency on input size, particularly in worst-case conditions.

B. Prefix-Oriented Indexing Approaches

To address the inefficiencies of repeated scanning, prefix-based indexing structures were introduced to exploit common leading characters among strings. Tries represent one of the most widely used realizations of this idea, organizing textual data as a hierarchical structure where each node corresponds to a character. Search operations in Tries are bounded by the length of the query pattern, enabling consistent performance independent of dataset size.

C. Suffix-Based Indexing and Substring Search

Prefix-oriented methods are inherently limited to queries that match from the beginning of a string. To support more general substring searches, suffix-based indexing techniques were developed. Suffix Trees enable linear-time pattern matching but are widely recognized as impractical for large-scale systems due to their substantial memory requirements.

D. Compressed and Advanced Index Structures

More recent research has explored compressed full-text indexing techniques aimed at further reducing memory consumption without sacrificing search performance. Structures such as FM-indexes and wavelet trees leverage data compression and succinct representations to support fast pattern matching on large text corpora. While these methods offer strong theoretical guarantees and improved space efficiency, they often involve complex implementation requirements and may introduce higher constant factors in practice.

E. Comparative Experimental Evaluations

Despite extensive research on individual text searching techniques, direct experimental comparisons conducted under uniform conditions remain relatively scarce. Many studies evaluate a single method in isolation or rely on theoretical complexity analysis to infer performance. Differences in implementation choices, datasets, and evaluation metrics further complicate meaningful comparison across studies.

This work contributes to the existing literature by providing a controlled empirical evaluation of sequential scanning, Trie-based indexing, and Suffix Array-based indexing within a

shared experimental framework. By focusing on observed runtime behavior, scalability trends, and memory trade-offs, the study aims to bridge the gap between theoretical analysis and practical system-level performance.

III. Methodology

This study adopts an experimental approach to evaluate the practical performance characteristics of commonly used text searching techniques under controlled conditions. The objective is to analyze how different search strategies behave as input size and query workload increase, with particular emphasis on execution efficiency, memory overhead, and scalability. Three representative approaches are considered: naive sequential search, Trie-based prefix indexing, and Suffix Array-based substring indexing. Together, these techniques capture a progression from scan-based matching to structured indexing methods.

A. Experimental Design

To ensure a fair and unbiased comparison, all evaluated algorithms were implemented using the same programming language and executed within a unified experimental framework. Python was selected due to its widespread use in prototyping and algorithmic evaluation, as well as its support for rapid development and performance measurement.

B. Dataset Preparation

The experiments employed a combination of synthetic and real-world textual datasets to capture diverse input characteristics. Dataset sizes ranged from 1,000 to 20,000 characters, representing small to moderately sized text collections commonly encountered in practical applications such as log analysis and document processing.

Prior to indexing or search execution, all datasets were preprocessed to ensure consistent formatting. This preprocessing included the removal of non-alphanumeric characters and conversion of all text to lowercase, reducing variability caused by character encoding and case sensitivity. These normalization steps ensure that observed performance differences are attributable to the search techniques rather than inconsistencies in input representation.

C. Implementation of Search Techniques

The three evaluated search techniques differ in their preprocessing requirements and query execution strategies:

- **Naive Sequential Search:** This approach performs direct pattern matching by aligning the query pattern at every possible position in the input text and comparing characters sequentially. No auxiliary data structures are constructed, resulting in negligible preprocessing overhead but increased query execution time as input size grows.
- **Trie-Based Prefix Search:** In this method, a prefix-oriented index is constructed during preprocessing by inserting characters into a Trie structure. Query execution involves traversing the Trie according to the characters of the search pattern. Since traversal depth depends on pattern length rather than dataset size, this approach offers stable query performance at the cost of increased memory usage.
- **Suffix Array-Based Substring Search:** The Suffix Array technique constructs a sorted index of suffix starting positions within the input text. During query execution, binary search is performed over the ordered suffix indices to determine the presence of the search pattern. This approach supports flexible substring matching while maintaining moderate memory requirements compared to tree-based alternatives.

D. Performance Metrics

The effectiveness of each search technique was evaluated using the following metrics:

- **Execution Time:** The total time required to process all search queries for a given dataset. Measurements include only query execution time, excluding dataset loading overhead.
- **Memory Usage:** The amount of memory consumed during index construction and query execution. This metric captures the space overhead introduced by auxiliary data structures.
- **Scalability:** The variation in execution time as dataset size increases, reflecting how well each technique adapts to growing input sizes.

E. Experimental Procedure

The experimental workflow followed a structured sequence of steps to ensure repeatability and consistency:

1) Normalize and preprocess the input text datasets.
2) Construct indexing structures where applicable.
3) Generate query patterns of varying lengths for each dataset.
4) Execute each search technique using identical datasets and query sets.
5) Measure execution time and memory consumption for each run.
6) Repeat experiments multiple times and compute average values to reduce the impact of transient system variability.
7) Compare results across all techniques using aggregated performance metrics.

F. Hardware and Software Environment

All experiments were conducted on a standard computing platform to reflect realistic deployment conditions. The experimental environment consisted of:

- Processor: Intel Core i5 or equivalent
- Memory: 8 GB RAM
- Operating System: Windows 10 / Linux
- Software Environment: Python 3.10 with standard libraries including `time`, `memory_profiler`, and `numpy`

G. Workflow Overview

Figure 2 illustrates the overall experimental workflow, summarizing the sequence of preprocessing, index construction, query execution, and performance measurement steps employed in this study.

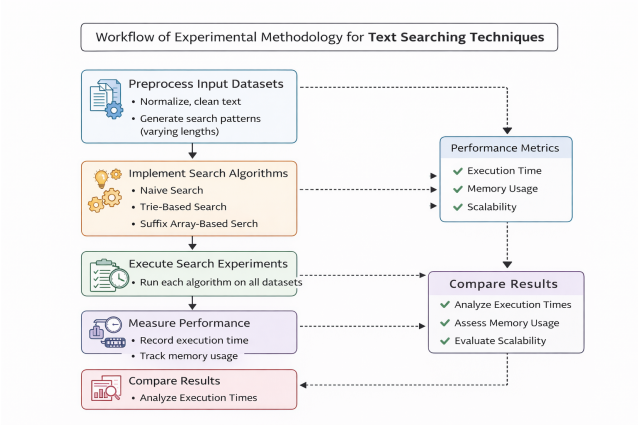

Fig. 2. Experimental workflow for evaluating text searching techniques

By maintaining consistent datasets, evaluation metrics, and execution conditions, the proposed methodology ensures that observed performance differences arise solely from the characteristics of the underlying search techniques. This structured experimental setup enables a reliable assessment of efficiency, scalability, and practical applicability in real-world text processing systems.

IV. Implementation Details

This section outlines the practical realization of the evaluated text searching techniques and discusses key design choices made during implementation. All algorithms were implemented in Python with a focus on clarity, consistency, and experimental reproducibility. Rather than employing aggressive low-level optimizations, the implementations emphasize transparent behavior that facilitates meaningful performance comparison across techniques.

A. Sequential Pattern Matching

The sequential pattern matching approach serves as a baseline against which indexed methods are compared. In this implementation, the search pattern is aligned at successive positions within the input text, and characters are compared sequentially until either a mismatch occurs or the entire pattern is matched. This process is repeated for every possible starting position in the text.

B. Trie-Oriented Search

The Trie-based search technique introduces a preprocessing phase in which a prefix-oriented index is constructed from the input dataset. In this study, the Trie is built once per dataset and reused across all query executions, allowing preprocessing costs to be amortized over multiple searches. Characters are inserted incrementally, forming a hierarchical structure in which shared prefixes correspond to common traversal paths.

Each node in the Trie represents a single character and maintains references to its child nodes. Terminal markers are used to indicate the completion of valid strings or prefixes. During query execution, the search pattern is processed character by character by traversing the corresponding nodes in the Trie. Successful traversal of all pattern characters indicates the existence of the queried prefix within the dataset.

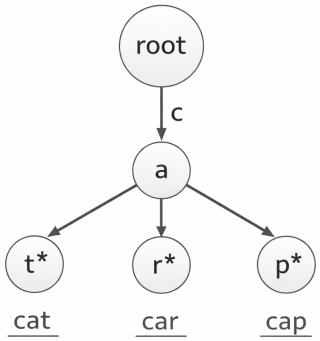

Fig. 3. Example of a Trie structure for the words “cat”, “car”, and “cap”. Terminal nodes indicate complete prefixes.

C. Suffix Array-Oriented Search

The Suffix Array-based approach supports flexible substring search by organizing the input text through an ordered index of suffix positions. During preprocessing, all suffixes of the text are conceptually generated, and their starting indices are sorted lexicographically. Instead of storing suffix strings explicitly, the implementation maintains integer references to positions within the original text, thereby reducing memory overhead.

Query execution is performed using binary search over the sorted suffix indices. At each step, the query pattern is compared with a corresponding substring of the text referenced by the current suffix index. This comparison process continues until a match is found or the search space is exhausted. Although suffix array construction introduces additional preprocessing cost, the resulting structure enables efficient substring matching across repeated queries.

D. Implementation Considerations and Trade-offs

Several practical considerations influenced the implementation of the evaluated techniques. Python’s object-oriented design simplifies the construction of complex data structures such as Tries but also introduces additional memory overhead due to object references. Conversely, the Suffix Array implementation relies primarily on integer indexing, resulting in improved cache locality and lower per-element storage costs.

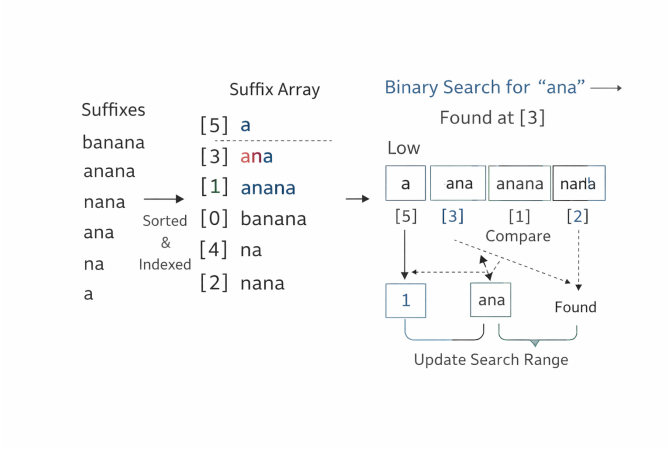

Fig. 4. Illustration of suffix array construction and binary search for the text "banana".

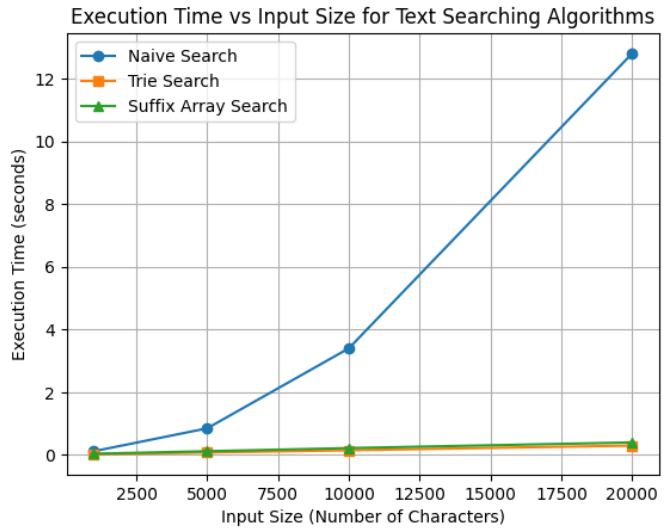

Fig. 5. Execution time versus input size for evaluated text searching techniques

E. Experimental Execution

Each experiment followed a consistent execution pipeline to ensure reproducibility. Input datasets were first normalized to enforce uniform character representation. For each dataset, multiple query patterns were generated and applied uniformly across all search techniques.

Execution time and memory usage were measured independently for each algorithm. To mitigate transient runtime variability introduced by background system processes, experiments were repeated multiple times and average values were used in subsequent analysis. This repetition ensures that observed performance trends accurately reflect the behavior of the underlying search strategies rather than incidental environmental factors.

V. Results and Discussion

This section presents and analyzes the experimental results obtained from evaluating the three text searching techniques across varying dataset sizes. The primary objective of the analysis is to examine how each approach responds to increasing input size and to identify practical trade-offs related to execution time, memory usage, and query flexibility.

A. Execution Time Analysis

Table I summarizes the measured execution times for naive sequential search, Trie-based search, and Suffix Array-based search for datasets containing 1K, 10K, and 20K characters. Figure 5 visualizes the corresponding trends, enabling direct comparison of scalability behavior.

TABLE I
Execution Time Comparison (in seconds)

Algorithm	1K	10K	20K
Naive Search	0.12	3.40	12.80
Trie Search	0.02	0.15	0.30
Suffix Array	0.04	0.22	0.40

The results reveal a pronounced divergence between scan-based and indexing-based approaches as dataset size increases. Sequential search exhibits a steep growth in execution time, with runtime increasing by more than an order of magnitude when input size scales from 1K to 20K characters. This behavior is a direct consequence of repeated character comparisons performed at each possible alignment position in the text.

B. Comparison of Index-Based Techniques

Among the evaluated indexing methods, Trie-based search consistently achieves the lowest execution time across all dataset sizes. This result can be attributed to the fact that Trie traversal is bounded by the length of the search pattern rather than the size of the input text. As a result, query latency remains stable even as the dataset grows, making Trie-based indexing particularly well suited for prefix-dominated search workloads.

Suffix Array-based search exhibits slightly higher execution times compared to Trie-based search. This difference arises from the use of binary search over the sorted suffix indices and repeated substring comparisons during query execution. However, the increase in execution time remains moderate and predictable, reflecting the logarithmic nature of the search process.

C. Scalability Trends

The scalability trends observed in Figure 5 further emphasize the contrasting behavior of the evaluated techniques. Sequential search shows near-linear growth in execution time relative to dataset size, indicating poor scalability under increasing workloads. In contrast, both Trie-based and Suffix Array-based approaches exhibit sublinear growth, reflecting their reliance on indexed access paths.

D. Memory and Performance Trade-offs

While execution time represents a critical performance metric, memory usage also plays an important role in determining the suitability of a search technique. Trie-based indexing incurs higher memory overhead due to the storage of node objects and character pointers, especially in datasets with limited prefix sharing. In contrast, Suffix Arrays store only integer references to text positions, resulting in lower per-element memory usage.

E. Implications for Practical Applications

From a practical perspective, the results underscore the importance of aligning search strategy selection with application requirements. Trie-based indexing is well suited for applications such as autocomplete systems, dictionary lookup, and lexical analysis, where prefix queries dominate and low-latency responses are critical. Suffix Array-based indexing is more appropriate for applications requiring general substring search, including document analysis, pattern detection, and bioinformatics workloads.

Overall, the results confirm that preprocessing-based text searching techniques provide substantial performance benefits over naive scanning approaches. The choice between Trie-based and Suffix Array-based methods should therefore be guided by query characteristics, memory constraints, and scalability requirements specific to the target application.

VI. Conclusion and Future Work

This work presented a systematic experimental evaluation of three representative text searching techniques—naive sequential scanning, Trie-based prefix indexing, and Suffix Array-based substring indexing—using a unified and controlled evaluation framework. By maintaining consistent datasets, implementations, and performance metrics, the study enabled direct comparison of practical trade-offs among these approaches.

The experimental results clearly demonstrate that preprocessing-based indexing techniques offer substantial performance advantages over scan-based matching as dataset size and query frequency increase. Sequential search, while simple and memory efficient, exhibits rapidly growing execution time and is therefore unsuitable for large-scale or query-intensive environments. In contrast, both Trie-based and Suffix Array-based approaches achieve significantly lower and more stable query latency by leveraging structured indexing mechanisms.

Trie-based search consistently delivers the fastest query response for prefix-oriented workloads, as traversal depth is bounded by query length rather than input size. This property makes Tries particularly well suited for applications such as autocomplete systems, dictionary lookup, and lexical analysis, where predictable response time is a primary requirement. However, this performance benefit comes at the cost of increased memory usage due to node and pointer storage.

Suffix Array-based search offers a more flexible alternative by supporting general substring matching while maintaining moderate memory overhead. Although query execution is slightly slower compared to Trie-based search, the ability to handle arbitrary patterns makes Suffix Arrays attractive for a broader range of text processing tasks, including document analysis, pattern discovery, and bioinformatics applications. The results indicate that Suffix Arrays provide a balanced compromise between query flexibility, memory consumption, and execution efficiency.

Future research directions include exploring memory-efficient variants of the evaluated indexing structures, such as compressed Tries and succinct suffix representations, to further reduce space overhead without sacrificing performance. Investigating parallel and incremental index construction techniques represents another promising direction, particularly for large-scale systems where datasets evolve over time. Additionally, extending the experimental evaluation to larger and more heterogeneous datasets, including multilingual corpora and source code repositories, would provide deeper insight into the robustness and generality of the observed performance trends.

Further work may also consider the impact of query characteristics, such as pattern length and frequency distribution, as well as the integration of approximate string matching techniques. Such extensions would enhance the applicability of the proposed evaluation framework and support informed selection of text searching strategies in diverse real-world computing environments.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.

[2] D. Gusfield, *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*, Cambridge University Press, 1997.

[3] U. Manber and G. Myers, "Suffix arrays: A new method for on-line string searches," *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.

[4] A. Aho and M. Corasick, "Efficient string matching: An aid to bibliographic search," *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.

[5] P. Ferragina and G. Manzini, "Opportunistic data structures with applications," in *Proc. 41st IEEE Symp. Foundations of Computer Science (FOCS)*, 2000, pp. 390–398.

[6] R. Navarro and V. Mäkinen, "Compressed full-text indexes," *ACM Computing Surveys*, vol. 39, no. 1, 2007.

[7] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.

[8] S. Gog and T. Beller, "From theory to practice: Plug and play with succinct data structures," in *Proc. 13th Int. Symp. Experimental Algorithms (SEA)*, 2014, pp. 326–337.

[9] G. Navarro, "Indexing highly repetitive string collections," *ACM Computing Surveys*, vol. 52, no. 2, 2019.

[10] J. Fischer, V. Heun, and T. I, "Practical applications of succinct data structures," *ACM Journal of Experimental Algorithmics*, vol. 26, 2021.

[11] S. Puglisi, W. Smyth, and A. Turpin, "A taxonomy of suffix array construction algorithms," *ACM Computing Surveys*, vol. 55, no. 3, 2023.